

Course Code: CS1083

Name of the Programme: BCA (Hons.)

Name of the Course: Data Structures using C

Semester: II

Course credit	No. of hours per week (L + T + P)	Total no. of Teaching hours
4	2+0+4	90
Course Objectives:		
a) Understand fundamental programming concepts essential for implementing data structures.		
b) Apply user-defined data types to implement linear data structures like linked lists, stacks, and queues.		
c) Implement binary trees and perform various traversal methods including inorder, preorder, and postorder.		
d) Analyze and apply tree-based data structures for efficient data storage and retrieval.		

Course outline (Syllabus of the course)

Syllabus:

Module - 1	No. of hours
	6
Foundations of Programming for data structures: Arrays, Pointers and Strings Functions and Recursion, Arrays, Pointers and Strings: Introduction to Arrays, Definition, One Dimensional Array and Multidimensional Arrays, Pointer, Pointer to Structure, various Programs for Array and Pointer. Strings. Introduction to Strings, Definition, Library Functions of Strings.	

Module - 2	No. of hours
	6
Implementing Data Structures with User-Defined Types: Linked Lists, Stacks, and Queues. Introduction to Data Structures, abstract data types, Stack ADT- definition and operations, Queue ADT- definition and operations, Linear list – Singly linked list implementation, insertion, deletion and searching operations on linear list.	

Module - 3	No. of hours
	6
Advanced Linked Lists: Implementation and Operations of Doubly and Circular Linked Circularly linked lists- Operations for Circularly linked lists, Doubly linked list implementation, insertion, deletion and searching operations, applications of linked lists	

Module - 4	No. of hours
	6
Binary Trees: Implementation and Traversal Techniques Trees – Definitions, tree representation, properties of trees, Binary tree, Binary tree representation, binary tree properties, binary tree traversals, binary tree implementation, applications of trees.	

Module - 5	No. of hours
	6
Graph Definition and properties of graphs, Types of graphs, Graph representations, Example of a basic graph implementation, Importance of graph traversal, BFS and DFS on Graph, Advantages and Disadvantages of Graph.	

Course outcomes
a) Understand the basic programming concepts needed to implement data structures.
b) Apply user-defined data types to implement linked lists, stacks, and queues.
c) Implement binary trees and perform different traversal methods like inorder, preorder, and postorder.
d) Analyze graph data structures, perform basic operations, and evaluate traversal techniques like BFS and DFS.

Text Books	
1	Karumanchi, N. (2016). Data structures and algorithms made easy (5th ed.). CareerMonk Publications. ISBN: 978-8193245279
2	Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1982). Data structures and algorithms (1st ed.). Addison-Wesley. ISBN: 978-0201000238

Reference books	
1	Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. ISBN: 978-0262046305
2	Sedgewick, R., & Wayne, K. (2011, with ongoing support). Algorithms (4th ed.). Addison-Wesley Professional. ISBN: 978-0321573513
3	Siddiqui, A. T., & Siddiqui, S. A. (2023). Data Structure Using C (1st ed.). CRC Press. ISBN: 978-1003453291

Reference links	
1.	https://www.geeksforgeeks.org/data-structures/

Lab Programs		[60 Hrs]
1	Write a program to find the largest and smallest element in an array.	
2	Write a program to find the factorial of a number using recursion. Write a program to find the nth Fibonacci number using recursion.	
3	Write a program to implement the Sudoku game structure and implement searching operations along rows, columns and grids.	
4	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Implement the following operations: 1. insert_at_beginning(data): Insert a new node with the given data at the beginning of the list. 2. insert_at_end(data): Insert a new node with the given data at the end of the list. 3. delete_node(data): Delete the first node in the list that contains the given data 4. traverse(): Traverse the list and print the data of each node	
5	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Stack to represent the stack itself. Implement the following operations: 1. push(data): Push a new node with the given data onto the stack. 2. pop(): Remove and return the top node from the stack. If the stack is empty, return None. 3. peek(): Return the data of the top node without removing it. If the stack is empty, return None 4. empty(): Return True if the stack is empty, otherwise return False	
6	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Queue to represent the queue itself. Implement the following operations: 1. enqueue(data): Add a new node with the given data to the end of the queue. 2. front(): Return the data of the front node without removing it. If the queue is empty, return None. 3. is_empty(): Return True if the queue is empty, otherwise return False	
7	Write a program to create a class/structure DoublyNode to represent a node in a doubly linked list. Each node should have three attributes: data, next, and prev. Then, create a class/structure DoublyLinkedList to represent the linked list itself.	

	Implement the following operations: 1. <code>insert_at_beginning(data)</code>: Insert a new node with the given data at the beginning of the list. 2. <code>insert_at_end(data)</code>: Insert a new node with the given data at the end of the list. 3. <code>traverse_forward()</code>: Traverse the list forward and print the data of each node.
8	Write a program to a class/structure <code>CircularNode</code> to represent a node in a circular linked list. Each node should have two attributes: <code>data</code> and <code>next</code>. Then, create a class/structure <code>CircularLinkedList</code> to represent the linked list itself. Implement the following operations: 1. <code>insert_at_beginning(data)</code>: Insert a new node with the given data at the beginning of the list. 2. <code>traverse()</code>: Traverse the list and print the data of each node.
9	Write a program to create a class/structure <code>TreeNode</code> to represent a node in a binary tree. Each node should have three attributes: <code>data</code>, <code>left</code>, and <code>right</code>. Then, create a class/structure <code>BinaryTree</code> to represent the binary tree itself. Implement the following operations: 1. <code>insert(data)</code>: Insert a new node with the given data into the binary tree (assume a simple insertion without balancing). 2. <code>pre_order_traversal()</code>: Perform a pre-order traversal of the tree and print the data of each node.
10	Write a program to create a class <code>TreeNode</code> to represent a node in a binary tree. Each node should have three attributes: <code>data</code>, <code>left</code>, and <code>right</code>. Then, create a class/structure <code>BinaryTree</code> to represent the binary tree itself. include a method/function <code>find_lca(node1, node2)</code> that takes two node values as input and returns the value of their lowest common ancestor. Assume all values in the tree are unique.
11	Write a program to create a class <code>TreeNode</code> to represent a node in a binary tree. Each node should have three attributes: <code>data</code>, <code>left</code>, and <code>right</code>. Then, create a class/structure <code>BinaryTree</code> to represent the binary tree itself, to include a method/function <code>find_grandchildren(node)</code> that takes a node value as input and returns a list of values of all the grandchildren of the given node.
12	Create a class/structure <code>Graph</code> to represent a graph using an adjacency list. Implement the following operations: 1. <code>add_edge(v, w)</code>: Add an edge between vertices <code>v</code> and <code>w</code>.
13	Create a class/structure <code>Graph</code> to represent a graph using an adjacency list, to include a method/function <code>dfs(start_vertex)</code> that performs a Depth-First Search starting from <code>start_vertex</code> and prints the order of traversal.
14	Create a class/structure <code>Graph</code> to represent an undirected graph using an adjacency list. Include a method/function <code>detect_cycle()</code> that detects if there is a cycle in the graph. The method should return <code>True</code> if a cycle is detected, otherwise <code>False</code>.
15	A social network is represented as an unweighted graph where users are vertices and friendships are edges. Implement a class/structure <code>SocialNetwork</code> that supports the following operations:

- | | |
|--|--|
| | <ol style="list-style-type: none">1. <code>add_friendship(user1, user2)</code>: Add a friendship (edge) between user1 and user2.2. <code>degrees_of_separation(user1, user2)</code>: Find the shortest path (in terms of the number of edges) between user1 and user2 using BFS and return the number of edges in the path. |
|--|--|

Name of the Programme:	BCA(H)	
Title of the Course:	Data Structures Using C	
Course code	CS1083	
Semester:	II	
Names of the Course Instructor(s)	Prof. Pranjul Shukla Prof. Manish Rajan	
Course credit	No. of hours per week (L + T + P)	Total no. of Teaching hours
4	2+0+4	90

Session	Module No.	Topic	Pedagogy /activities	Pre-reading/ reference
1	1	Functions and Recursion: Basics	Chalk & Talk, Board Tracing	Text Book 1 (Karumanchi) ¹
2	1	Recursion Implementation Logic	Live Coding, Dry Run	Reference Link 1 (GFG) ²
3	1	Lab: Logic for Array Min/Max	Logic Building, Dry-run	Syllabus Lab Program 1 ³
4	1	Lab: Execution of Array Min/Max	Coding, Debugging	Syllabus Lab Program 1 ⁴

5	1	Lab: Logic for Factorial (Recursion)	Logic Building, Dry-run	Syllabus Lab Program 2 ⁵
6	1	Lab: Execution of Fibonacci (Recursion)	Coding, Debugging	Syllabus Lab Program 2 ⁶
7	1	Introduction to Arrays: 1D & 2D	Visual Representation	Text Book 2 (Aho) ⁷
8	1	Pointers and Pointer to Structure	Memory Diagram Tracing	Text Book 1 (Karumanchi) ⁸
9	1	Lab: Sudoku Game Structure Setup	Case Study, Coding	Syllabus Lab Program 3 ⁹
10	1	Lab: Sudoku Row/Column Search	Coding, Dry-run	Syllabus Lab Program 3 ¹⁰
11	1	Lab: Sudoku Grid Searching Logic	Coding, Debugging	Syllabus Lab Program 3 ¹¹
12	1	Lab: Sudoku Final Execution	Testing, Execution	Syllabus Lab Program 3 ¹²

13	1	Strings Intro & Library Functions	Demonstration, Examples	Reference Link 2 (Scaler) ¹³
14	1	Pointer Arithmetic & Structures	Board Tracing, Live coding	Ref Book 3 (Siddiqui) ¹⁴
15	2	Lab: Singly Linked List (SLL) Node Logic	Logic Building, Coding	Syllabus Lab Program 4 ¹⁵
16	2	Lab: SLL: Insert at Beginning/End	Coding, Dry-run	Syllabus Lab Program 4 ¹⁶
17	2	Lab: SLL: Delete Node Implementation	Coding, Debugging	Syllabus Lab Program 4 ¹⁷
18	2	Lab: SLL: Traversal and Printing	Testing, Execution	Syllabus Lab Program 4 ¹⁸
19	2	Intro to Data Structures & ADT Concept	Concept discussion, Analogies	Text Book 1 (Karumanchi) ¹⁹
20	2	Stack ADT: Definition & Operations	Visual simulation, Diagrams	Text Book 1 (Karumanchi) ²⁰

21	2	Lab: Stack Node Structure	Logic Building, Coding	Syllabus Lab Program 5 ²¹
22	2	Lab: Stack: Push & Pop Implementation	Coding, Dry-run	Syllabus Lab Program 5 ²²
23	2	Lab: Stack: Peek & Empty Checks	Coding, Debugging	Syllabus Lab Program 5 ²³
24	2	Lab: Stack: Testing with Edge Cases	Testing, Execution	Syllabus Lab Program 5 ²⁴
25	2	Queue ADT: Definition & Operations	Analogies, Diagrams	Text Book 2 (Aho) 25
26	2	Singly Linked List: Insertion Logic	Diagram-based teaching	Text Book 1 (Karumanchi) ²⁶
27	2	Lab: Queue Node Structure	Logic Building, Coding	Syllabus Lab Program 6 ²⁷
28	2	Lab: Queue: Enqueue Implementation	Coding, Dry-run	Syllabus Lab Program 6 ²⁸

29	2	Lab: Queue: Front & Is_Empty	Coding, Debugging	Syllabus Lab Program 6 ²⁹
30	2	Lab: Queue: Final Testing	Testing, Execution	Syllabus Lab Program 6 ³⁰
31	2	SLL: Deletion & Searching Operations	Live Coding, Trace list	Reference Link 1 (GFG) ³¹
32	2	SLL: Review of ADT Operations	Quiz, Discussion	Text Book 1 (Karumanchi) ³²
33	3	Lab: Doubly Linked List (DLL) Node Logic	Logic Building, Coding	Syllabus Lab Program 7 ³³
34	3	Lab: DLL: Insert Beginning/End	Coding, Dry-run	Syllabus Lab Program 7 ³⁴
35	3	Lab: DLL: Forward Traversal	Coding, Debugging	Syllabus Lab Program 7 ³⁵
36	3	Lab: DLL: Testing & Validation	Testing, Execution	Syllabus Lab Program 7 ³⁶

37	3	Circular Linked List Operations	Visual Diagram Tracing	Ref Book 1 (Cormen) ³⁷
38	3	Doubly Linked List Implementation	Board Walkthrough	Text Book 1 (Karumanchi) ³⁸
39	3	Lab: Circular Node Structure	Logic Building, Coding	Syllabus Lab Program 8 ³⁹
40	3	Lab: Circular LL: Insert Beginning	Coding, Dry-run	Syllabus Lab Program 8 ⁴⁰
41	3	Lab: Circular LL: Traversal Logic	Coding, Debugging	Syllabus Lab Program 8 ⁴¹
42	3	Lab: Circular LL: Final Testing	Testing, Execution	Syllabus Lab Program 8 ⁴²
43	3	DLL: Insertion and Deletion Logic	Chalk & Talk, Live code	Text Book 1 (Karumanchi) ⁴³
44	3	DLL: Searching and Applications	Dry Run on Board	Text Book 2 (Aho) ⁴⁴

45	4	Lab: Binary Tree Node Class	Logic Building, Coding	Syllabus Lab Program 9 ⁴⁵
46	4	Lab: Binary Tree: Simple Insertion	Coding, Dry-run	Syllabus Lab Program 9 ⁴⁶
47	4	Lab: Binary Tree: Pre-order Traversal	Coding, Debugging	Syllabus Lab Program 9 ⁴⁷
48	4	Lab: Binary Tree: Execution	Testing, Execution	Syllabus Lab Program 9 ⁴⁸
49	3	Applications of Linked Lists	Case-Based Learning	Text Book 1 (Karumanchi) ⁴⁹
50	3	Review of Advanced Linked Lists	Physical Simulation	Ref Book 2 (Sedgewick) ⁵⁰
51	4	Lab: LCA: Search Logic Design	Logic Building, Dry-run	Syllabus Lab Program 10 ⁵¹
52	4	Lab: LCA: Recursive Implementation	Coding, Debugging	Syllabus Lab Program 10 ⁵²

53	4	Lab: LCA: Unique Value Assumption	Testing, Verification	Syllabus Lab Program 10 ⁵³
54	4	Lab: LCA: Final Code Testing	Execution, Debugging	Syllabus Lab Program 10 ⁵⁴
55	4	Tree Definitions & Properties	Diagrams, Hierarchies	Text Book 1 (Karumanchi) ⁵⁵
56	4	Tree Representation & Applications	Chalk & Talk, Exercises	Text Book 1 (Karumanchi) ⁵⁶
57	4	Lab: Grandchildren Search Logic	Logic Building, Dry-run	Syllabus Lab Program 11 ⁵⁷
58	4	Lab: Grandchildren: List Implementation	Coding, Debugging	Syllabus Lab Program 11 ⁵⁸
59	4	Lab: Grandchildren: Traversal Checks	Testing, Verification	Syllabus Lab Program 11 ⁵⁹
60	4	Lab: Grandchildren: Final Debugging	Execution, Debugging	Syllabus Lab Program 11 ⁶⁰

61	4	Binary Tree Representation	Board Tracing, Maps	Text Book 2 (Aho) 61
62	4	Binary Tree Properties	Visual Walkthrough	Text Book 1 (Karumanchi) 62
63	5	Lab: Graph Adjacency List Setup	Logic Building, Coding	Syllabus Lab Program 12 63
64	5	Lab: Graph: Add Edge (v, w)	Coding, Dry-run	Syllabus Lab Program 12 64
65	5	Lab: Graph: Pointer Management	Coding, Debugging	Syllabus Lab Program 12 65
66	5	Lab: Graph: Final Display Logic	Testing, Execution	Syllabus Lab Program 12 66
67	4	Binary Tree Traversals (In/Pre/Post)	Think–Pair–Share, Tracing	Ref Book 3 (Siddiqui) 67
68	4	Binary Tree Implementation	Live Coding, Group tracing	Reference Link 1 (GFG) 68

69	5	Lab: DFS Logic (Stack/Recursion)	Logic Building, Dry-run	Syllabus Lab Program 13 ⁶⁹
70	5	Lab: DFS Implementation in C	Coding, Debugging	Syllabus Lab Program 13 ⁷⁰
71	5	Lab: DFS: Printing Order	Testing, Verification	Syllabus Lab Program 13 ⁷¹
72	5	Lab: DFS: Traversal Testing	Execution, Debugging	Syllabus Lab Program 13 ⁷²
73	4	Binary Tree Review & Summary	Real-world Demo	Text Book 1 (Karumanchi) ⁷³
74	5	Graph: Definitions & Properties	Diagrams, Network Maps	Text Book 1 (Karumanchi) ⁷⁴
75	5	Lab: Graph Cycle Detection Logic	Logic Building, Dry-run	Syllabus Lab Program 14 ⁷⁵
76	5	Lab: Cycle Detect Implementation	Coding, Debugging	Syllabus Lab Program 14 ⁷⁶

77	5	Lab: Boolean Return Checks	Testing, Verification	Syllabus Lab Program 14 ⁷⁷
78	5	Lab: Cycle Testing on Graphs	Execution, Debugging	Syllabus Lab Program 14 ⁷⁸
79	5	Types of Graphs & Representation	Chalk & Talk, Mapping	Text Book 2 (Aho) 79
80	5	Graph Implementation Example	Board Diagrams, Walkthrough	Text Book 1 (Karumanchi) ⁸⁰
81	5	Lab: Social Network: Add Friend	Logic Building, Coding	Syllabus Lab Program 15 ⁸¹
82	5	Lab: Social Network: BFS Shortest Path	Coding, Dry-run	Syllabus Lab Program 15 ⁸²
83	5	Lab: Social Network: Degree Logic	Coding, Debugging	Syllabus Lab Program 15 ⁸³
84	5	Lab: Social Network: BFS Testing	Testing, Execution	Syllabus Lab Program 15 ⁸⁴

85	5	Importance of Graph Traversal	Live Coding, Case study	Reference Link 2 (Scaler) ⁸⁵
86	5	BFS and DFS on Graph	Concept Discussion, Tracing	Ref Book 1 (Cormen) ⁸⁶
87	5	Advantages & Disadvantages of Graphs	Visual Tracing, Discussion	Ref Book 2 (Sedgewick) ⁸⁷
88	5	Module 5 Recap & Graph Summary	Board Discussion, Q&A	Text Book 1 (Karumanchi) ⁸⁸
89	-	Revision: Practical Exam Prep	Mock Lab, Debugging	Lab Programs 1-15 ⁸⁹
90	-	Revision: Final Course Review	Discussion, Quiz	All Course Outcomes ⁹⁰

Assessment and Evaluation

Continuous Internal Evaluation (CIE)	
Components of CIE	Weightage of each component
CIE-1: Quiz	10 marks out of 70
CIE-1: Scenario-Based Test	10 marks out of 70

CIE 2:Pen Paper Mode	25 marks out of 70
CIE 3: Practical Test +Viva	20 marks out of 70
CIE 3: Github Log	5 marks out of 70

Semester End Exams (SEE)	
Mode of Exam	Practical
Weightage of exam	30

Course Outcomes- Programme Outcomes Matrix

Sl.No	Course outcome	Programme Outcome											
		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
1	CO1: Understand basic programming concepts for data structures	3	2	1	2	1	1	1	2	1	1	1	1
2	CO2: Apply user-defined data types to implement stacks, queues & linked lists	3	3	2	3	1	2	2	2	1	1	1	1
3	CO3: Implement binary trees and traversal methods	3	3	2	3	1	2	2	2	1	1	1	1

4	CO4: Analyze graph structures and traversal technique s (BFS/DF S)	3	3	3	3	1	2	2	3	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---

Rubrics for evaluation of CIE

Total Marks		20
CIE-1 Components		Rubrics for Assessment
CIE-1	Quiz	One mark questions will be conducted for 10 marks. Each right answer carries one mark. No negative marking for wrong answers.
	Graded Component-1	
	Scenario Based Test	3 Questions for 10 marks First Question:5 marks out of 10 Second Question: 3 marks out of 10 Third question: 2 marks out of 10
	Graded Component-2	

Total Marks		25
CIE-2 Components		Rubrics for Assessment
CIE-2	Theory	The test will be conducted for 25 marks and will be evaluated according to the Scheme of Evaluation prepared by the team of course faculty.

Total Marks		25
CIE-3 Components		Rubrics for Assessment
CIE-3	Lab Program Submission + Viva-Voce Graded Component-1	Lab Program + Viva-Voce = 5 marks (<i>Refer Table-1</i>)
	Practical Test Graded Component-2	- Write-up:12 marks out of 20 (2 programs,each program carries 6 marks) - Execution:4 marks out of 20 (1 program execution) - Viva-Voce:4 marks out of 20 (<i>Refer Table-2</i>) Total:20

Note:Total Lab Program Submission + Viva-Voce: 15 programs each carries 10 marks which sums up to 150 marks and will be down scaled to 5 marks.

Total Marks		30
SEE Components		Rubrics for Assessment
SEE	Practical	Write-up:20 marks out of 30(2 programs) Execution: 5 marks out of 30(1 program execution) Viva-Voce: 5 marks out of 30 (<i>Refer Table-3</i>)

Table-1

Practical Component Evaluation Rubrics: Graded Component 1

Lab Program Submission (GoogleClassRoom) + Viva (10 Marks)

Lab Program Submission (GoogleClassRoom):6Marks

Sl. No	Criteria	Measuring Method	Excellent	Very Good	Good	Average	Needs Improvement
1	Code Quality & Logic(3)	Review of code in Google	Code is efficient, well-structur	Minor issues but overall correct and	Logic partially correct;	Code runs with multiple	Major issues; code does not run

		Classroom submission	ed, uses correct logic; no errors(3)	efficient(2)	minor errors (1)	errors; logic unclear (0.5)	(0)
2	Output Accuracy(2)	Program execution and screenshots	Accurate output for all test cases (2)	Minor deviations in output (2)	Correct for basic cases only (1)	Output partially correct (0.5)	Output incorrect (0)
3	Timely Submission(1)	Timestamp in Google Classroom	Submitted before deadline (1)	Submitted on deadline (0.8)	Submitted within 12 hours after deadline (0.6)	Submitted within 24 hours after deadline (0.4)	Not submitted (0)

Viva-Voce:4Marks

One mark questions will be conducted for 4 marks. Each right answer carries one mark. No negative marking for wrong answers.

Table-2

Practical Component Evaluation Rubrics: Graded Component 2

Sl. No	Criteria	Measuring Method	Excellent	Very Good	Good	Average	Needs Improvement
1	Program Write-up (per program 6M) (12M)	Review submitted document (PDF/notebook)	Complete and accurate problem statement, algorithm, and code; well-organized (6)	Minor omissions or formatting issues; logic mostly correct (5)	Partial write-up; minor errors in logic (3-4)	Weak write-up; incomplete algorithm/code (2)	Missing or incorrect write-up (1)
2	Program Execution & Output (1 program execution) (4M)	Instructor observes live execution or reviews recorded output	Code runs successfully; outputs correct results without guidance (4)	Minor errors; outputs mostly correct; little guidance required (3)	Runs with partial correctness; some guidance needed (2)	Runs with errors; significant guidance needed (1)	Code does not run(0)
3	Conceptual	Oral questioning	Clear,	Good	Basic	Weak	Cannot

	1 Understanding & Explanation (4M)	during viva	confident, complete understanding; can explain logic, algorithms, and output (4)	understanding; minor gaps; mostly clear explanation (3)	understanding; partially correct explanation (2)	understanding; explanations unclear (1)	explain; no understanding (0)
--	---	-------------	---	---	---	--	-------------------------------------

Table-3
Semester End Examinations

Sl. No	Criteria	Measuring Method	Excellent	Very Good	Good	Average	Needs Improvement
1	Program Write-up (per program 10 M) (20M)	Review submitted document (PDF/notebook)	Complete and accurate problem statement, algorithm, and code; well-organized (9-10)	Minor omissions or formatting issues; logic mostly correct (7-8)	Partial write-up; minor errors in logic (5-6)	Weak write-up; incomplete algorithm/code (3-4)	Missing or incorrect write-up (0-2)
2	Program Execution & Output (1 program execution) (5M)	Instructor observes live execution or reviews recorded output	Code runs successfully; outputs correct results without guidance (5)	Minor errors; outputs mostly correct; little guidance required (4)	Runs with partial correctness; some guidance needed (3-2)	Runs with errors; significant guidance needed (1)	Code does not run(0)
3	Conceptual Understanding & Explanation	Oral questioning during viva	Clear, confident, complete understanding; can	Good understanding; minor gaps;	Basic understanding; partially correct explanation	Weak understanding; explanations unclear	Cannot explain; no understanding (0)

	n (5M)		explain logic, algorithms, and output (5)	mostly clear explanatio n (4)	(3-2)	(1)	
--	-----------	--	---	---	-------	-----	--

Signature of the Course Faculty